Build Prompt — Duka Lite (Simple POS)

> Paste this whole file as the opening instruction to your coding agent (Claude Code, Cursor, etc.). Do not let the agent expand scope. Anything not listed under **In Scope** is explicitly forbidden in v1.

Role

You are a senior full-stack engineer building a production-ready, offline-first Point of Sale web app for small retail shops in Kenya. The users are shop attendants with low digital literacy, working on cheap Android phones and low-end laptops, on unreliable 3G. Speed of checkout and clarity of the end-of-day numbers matter more than features.

The problem you are solving

A shop owner cannot reconcile their day. Cash in the drawer and money received via M-Pesa get mixed up in one mental total. At closing time the owner cannot tell whether a shortfall is a keying error or an

attendant who pocketed cash and claimed the customer paid by M-Pesa. Every screen you build should serve that one outcome: **at any moment, the owner knows exactly how much should be in cash and how much should be in M-Pesa.**

Stack (do not substitute)

- Next.js (App Router) + TypeScript
- Tailwind CSS
- Supabase (Postgres + Auth), with Row Level Security
- PWA: installable, works fully offline, syncs when connection returns
- Local write store: IndexedDB (use Dexie.js)
- Deploy target: Vercel

In Scope — v1 only

1. Auth and tenancy

- Email + password login via Supabase Auth.
- Two roles: `owner` and `attendant`.
- Every table carries a `shop_id`. RLS must enforce that a user only ever reads or writes rows for their own shop. Write the RLS policies explicitly; do not rely on client-side filtering.

2. Products / Stock

- Fields: name, unit (pc, kg, litre, packet), buying price, selling price, current quantity, low-stock threshold.
- Add, edit, deactivate a product. Never hard-delete — soft delete with `is_active`.
- Stock movements table recording every change with a reason: `sale`, `restock`, `adjustment`, `damage`. Quantity on the product is derived from movements, never edited directly.
- A restock screen: pick product, enter quantity received and buying price.
- A low-stock list showing everything at or below its threshold.

3. Sell screen (the most important screen)

- Big, tappable product tiles with a search box at the top. Optimised for one-handed thumb use.
- Tap a product to add to the current sale; tap again to increment. Allow quantity edit and line removal.
- Running total shown large and always visible.
- Checkout asks one question only: **Cash or M-Pesa?** Two large buttons, visually distinct colours, no dropdown.
    - If Cash: prompt for amount tendered, display change due in large type.

    - If M-Pesa: prompt for the M-Pesa confirmation code (e.g. `SGH7X2K9LM`). Accept it as a free-text string, store it, but do not validate against Safaricom — there is no API integration in v1.
- On confirm: write the sale, write the sale lines, write the stock movements, clear the cart. The whole thing must complete in under one second and must work with no internet.

4. Reconciliation — the hero screen

A single page titled **Closing** showing, for the selected day:

Cash sales	KES ___
M-Pesa sales	KES ___
Total sales	**KES ___**
Number of transactions	___

Below that, a cash count box where the attendant enters the actual cash counted in the drawer, and the app shows **Expected vs Counted vs Difference**, with the difference in green when zero and red when not.

Below that, the list of M-Pesa codes recorded for the day so the owner can tick them off against the Safaricom statement by eye.

Default the date to today; allow picking any past date. This screen must be printable and shareable as a plain-text summary via the Web Share API (so it can be sent on WhatsApp).

5. Offline behaviour

- All reads come from IndexedDB. The app never blocks on the network.
- All writes go to IndexedDB first, then queue for sync to Supabase.
- Sales carry a client-generated UUID so re-sync is idempotent. Syncing the same sale twice must never double-count.
- A persistent, unobtrusive indicator: **Online / Offline · N pending**.
- Never show a spinner that blocks checkout.

Explicitly OUT of scope — do not build these

Credit sales or customer debt · customers or loyalty · suppliers or purchase orders · barcode scanning · multi-branch · discounts or promotions · returns or refunds · VAT or eTIMS · M-Pesa Daraja / STK Push ·

receipt printers · charts and dashboards · dark mode · multi-currency · CSV import/export · SMS.

If you believe one of these is necessary, stop and state your reasoning instead of building it.

UI rules

- Every label in English with the common Kiswahili term where it helps: *Uza* (Sell), *Bidhaa* (Products), *Kufunga* (Closing).
- Currency always formatted as `KES 1,250` — comma-separated thousands, no decimals.
- Minimum tap target 48px. Body text minimum 16px.
- Cash and M-Pesa must have consistent, distinct colours everywhere they appear. Never mix them into a single figure without also showing the split.
- No modals during checkout.

Data model

Give me these tables with sensible constraints and indexes:

`shops`, `users` (extending auth.users with shop_id + role), `products`, `stock_movements`, `sales`, `sale_items`, `cash_counts`.

`sales` must include: `id` (client UUID), `shop_id`, `user_id`, `total`, `payment_method` (enum: `cash` | `mpesa`), `amount_tendered` (nullable), `mpesa_code` (nullable), `created_at`, `synced_at`.

Add a check constraint: a `cash` sale must have `amount_tendered` and null `mpesa_code`; an `mpesa` sale must have `mpesa_code` and null `amount_tendered`.

How to work

1. First, output the complete SQL migration (schema + RLS policies) and stop for my review.
2. Then scaffold the project and the offline data layer.
3. Then build screens in this order: Login → Products → Sell → Closing.
4. After each screen, tell me what you built and what you deliberately left out.
5. Seed the database with 20 realistic Kenyan duka products (Unga Jogoo 2kg, Omo 500g, Blue Band 250g, Kasuku 500ml, Sukari 1kg, etc.) so I can test immediately.

Definition of done

I can, on a phone with aeroplane mode on: log in, sell three items paid in cash, sell two items paid by M-Pesa, see stock drop correctly, open the Closing screen, see cash and M-Pesa separated, enter a cash count, see the difference calculated — then turn the internet back on and watch everything sync exactly once.